

[Proyectos](#) · [Conceptos](#) · [Plantillas de código](#)

[Conceptos OOP](#) · [Colecciones](#) · [Arrays](#) · [Strings](#) · [LocalDate](#)
[Ficheros](#) · [JDBC](#) · [Algoritmos](#) · [Validación](#) · [Lambda](#)

[Curso 2025–2026](#) · [Java](#)

SECCIÓN 1 — CONCEPTOS CLAVE POR TEMA

1.1 ORIENTACIÓN A OBJETOS (OOP)

Clase abstracta

```
public abstract class Persona {
    private String nombre;
    public Persona(String nombre) { this.nombre = nombre; }
    public String getNombre() { return nombre; }
    public abstract String getDatos(); // obliga a implementar en hijos
}
```

→ Ejemplos: *Persona.java* (Badminton/Futbol/Médicos/Empresa), *Vehiculo.java* (Concesionario), *Artista.java* (Festival), *Empleados.java* (Examen27Enero)

Herencia

```
public class Jugador extends Persona {
    private int dorsal;
    public Jugador(String nombre, int dorsal) {
        super(nombre); // llama al constructor del padre
        this.dorsal = dorsal;
    }
    @Override
    public String getDatos() { return getNombre() + ' #' + dorsal; }
}
```

→ Ejemplos: *Arbitros*, *Jugadores*, *Entrenadores* (extienden *Persona*); *Coche/Moto* (extienden *Vehiculo*)

Interfaces

```
public interface Jubilacion {
    int EDAD_JUBILACION = 65; // constante (implicitamente public static final)
    void consultaJubilacion(); // método abstracto
}
public class Persona implements Baja, Jubilacion, Accidente {
    @Override public void consultaJubilacion() { ... }
}
```

→ Ejemplos: *Baja/Jubilacion/Accidente* (*gestionPersona3aev*), *Operacion.java* (*funcionesLambda*)

Interfaz funcional + Lambda

```
@FunctionalInterface
public interface Operacion {
    int ejecutar(int a, int b);
}
// Uso con lambda:
Operacion suma = (a, b) -> a + b;
Operacion resta = (a, b) -> a - b;
System.out.println(suma.ejecutar(5, 3)); // → 8
```

→ Ejemplo: *Operacion.java* (*funcionesLambda*)

Constructor overloading & Static counter para código

```
private static int contador = 0;
private String codigo;

public Artista(String nombre) { // constructor con 1 param
    contador++;
    this.codigo = String.format('ART-%03d', contador);
    this.nombre = nombre;
}
public Artista(String nombre, double sueldo) { // overloading
    this(nombre); // llama al constructor anterior
    this.sueldo = sueldo;
}
```

→ Ejemplos: *Artista.java* (Festival), *Vuelo.java* (Aeropuerto), *Empleados.java* (Examen27Enero), *Cuentas.java* (Banco)

1.2 COLECCIONES

ArrayList (orden, permite duplicados)

```
import java.util.ArrayList;
ArrayList<String> lista = new ArrayList<>();
```

```

lista.add('elemento'); // añadir al final
lista.add(0, 'primero'); // añadir en posición
lista.get(i); // obtener por índice
lista.remove(0); // eliminar por índice
lista.remove('elemento'); // eliminar por objeto
lista.size(); // número de elementos
lista.contains('algo'); // true si existe
lista.set(i, nuevoValor); // reemplazar en posición i
for (String s : lista) { ... } // recorrer con for-each

```

→ Ejemplos: *Clubes.java*, *Banco.java*, *Vehiculo.java* (Transportes), *Juego.java* (Examen26Feb), *Vuelo.java*, *Tarea.java*

HashSet (sin duplicados, sin orden)

```

import java.util.HashSet;
HashSet<Jugador> jugadores = new HashSet<>();
jugadores.add(jugador); // añadir (ignora si ya existe)
jugadores.remove(jugador); // eliminar
jugadores.contains(jugador); // buscar
jugadores.size(); // tamaño

```

→ Ejemplos: *Clubes.java* (Badminton), *Equipo.java* (Fútbol)

HashMap (clave → valor)

```

import java.util.HashMap;
HashMap<String, Integer> votos = new HashMap<>();
votos.put('candidato', 0); // insertar/actualizar
votos.get('candidato'); // obtener valor
votos.containsKey('candidato'); // true si existe la clave
votos.put(k, votos.get(k) + 1); // incrementar contador
for (String k : votos.keySet()) { ... } // recorrer claves

```

→ Ejemplo: *analisisVotosHashMap.java* (practicaExamenOrdinaria)

1.3 ARRAYS

```

// Array 1D
int[] arr = new int[5]; // declarar con tamaño
int[] arr = {1, 2, 3, 4, 5}; // declarar con valores
arr[0] = 10; // asignar
arr.length; // longitud
for (int i = 0; i < arr.length; i++) { arr[i]; }
for (int x : arr) { System.out.println(x); } // for-each

// Array 2D (matrices)
int[][] tablero = new int[filas][cols];
tablero[i][j] = valor;
for (int i=0; i<tablero.length; i++) {
for (int j=0; j<tablero[i].length; j++) { tablero[i][j]; }
}

```

→ Array 1D: *Contador_de_vocales.java*, *analisisVotos.java*, *Ejercicio_1.java* (Examen27Nov)

→ Array 2D: *Buscaminas.java*

SECCIÓN 1 (continuación) — STRINGS, FECHAS Y MÁS

1.4 STRINGS — MÉTODOS ESENCIALES

Método	Descripción	Ejemplo
<code>.equals(s)</code>	Compara contenido (no <code>==</code>)	<code>"hola".equals("hola") → true</code>
<code>.equalsIgnoreCase(s)</code>	Sin distinción mayús/minús	<code>"Hola".equalsIgnoreCase("hola")</code>
<code>.compareTo(s)</code>	Orden alfa (neg/0/pos)	<code>"a".compareTo("b") → -1</code>
<code>.toLowerCase()</code>	Todo en minúsculas	<code>"JAVA".toLowerCase() → "java"</code>
<code>.toUpperCase()</code>	Todo en mayúsculas	<code>"java".toUpperCase() → "JAVA"</code>
<code>.trim()</code>	Elimina espacios extremos	<code>" hi ".trim() → "hi"</code>
<code>.isEmpty()</code>	True si cadena vacía	<code>"".isEmpty() → true</code>
<code>.charAt(i)</code>	Carácter en posición i	<code>"java".charAt(1) → a</code>
<code>.length()</code>	Longitud de la cadena	<code>"hola".length() → 4</code>
<code>.split(sep)</code>	Divide en array de Strings	<code>"a,b".split(",") → ["a","b"]</code>
<code>.contains(sub)</code>	True si contiene subcadena	<code>"java".contains("av") → true</code>
<code>.substring(i,j)</code>	Subcadena desde i hasta j	<code>"java".substring(1,3) → "av"</code>
<code>String.format(fmt,...)</code>	Formatea con patrón	<code>String.format("%05d", 3) → "00003"</code>

→ Ejemplos: *Boletin_2 (compareTo)*, *Contador_de_vocales (charAt/length)*, *Profesor.java (equalsIgnoreCase/toUpperCase)*

1.5 FECHAS — LocalDate

```
import java.time.LocalDate;
import java.time.Period;
import java.time.format.DateTimeFormatter;

LocalDate hoy = LocalDate.now();
LocalDate nacimiento = LocalDate.of(2004, 5, 15);
int edad = Period.between(nacimiento, hoy).getYears();

DateTimeFormatter fmt = DateTimeFormatter.ofPattern('dd/MM/yyyy');
String fechaStr = hoy.format(fmt); // '26/05/2026'
LocalDate parsed = LocalDate.parse('26/05/2026', fmt); // texto a fecha
```

→ Ejemplos: *Jugadores.java (Badminton)*, *Cita.java (Médicos)*, *Persona.java (gestionPersona3ªev)*

1.6 FICHEROS (File I/O)

Leer con `BufferedReader` (try-with-resources) ← MÁS COMÚN

```
import java.io.*;

try (BufferedReader br = new BufferedReader(new FileReader('datos.txt'))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
        String[] partes = linea.split(','); // dividir por coma
    }
} catch (IOException e) { e.printStackTrace(); }
```

Leer con `Scanner`

```
Scanner sc = new Scanner(new File('datos.txt'));
while (sc.hasNextLine()) {
    String linea = sc.nextLine();
}
sc.close();
```

Leer con `Files API` (más moderno)

```
import java.nio.file.Files; import java.nio.file.Path;
List<String> lineas = Files.readAllLines(Path.of('datos.txt'));
String todo = Files.readString(Path.of('datos.txt'));
```

Escribir con PrintWriter

```
try (PrintWriter pw = new PrintWriter(new FileWriter('datos.txt'))) {  
    pw.println('linea 1');  
    pw.printf('%s,%d%n', nombre, valor);  
} catch (IOException e) { e.printStackTrace(); }
```

Ficheros binarios (Serialización)

```
// Clase debe implementar Serializable  
// Escribir:  
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream('f.dat'));  
oos.writeObject(objeto); oos.close();  
// Leer:  
ObjectInputStream ois = new ObjectInputStream(new FileInputStream('f.dat'));  
MiClase obj = (MiClase) ois.readObject(); ois.close();
```

→ Ejemplos: *Ejercicio1.java* (Boletin23), *ficheros.java* (manejodeficheros), *Tarea.java* (ListaTareas)

SECCIÓN 1 (continuación) — BBDD, ALGORITMOS Y VALIDACIÓN

1.7 BASES DE DATOS — JDBC

```
import java.sql.*;

try {
    Connection con = DriverManager.getConnection(
        'jdbc:mysql://localhost/nombreBD', 'usuario', 'contrasena');

    // Consulta con parámetro (seguro contra SQL injection)
    PreparedStatement ps = con.prepareStatement(
        'SELECT * FROM empleados WHERE ciudad = ?');
    ps.setString(1, 'Madrid'); // ps.setInt(1, 5); para enteros

    ResultSet rs = ps.executeQuery();
    while (rs.next()) {
        String nombre = rs.getString('nombre');
        int edad = rs.getInt('edad');
        System.out.println(nombre + ' - ' + edad);
    }
    rs.close(); ps.close(); con.close();

} catch (SQLException e) { e.printStackTrace(); }
```

→ Ejemplos: BasesDeDatos.java, EjercicioLogin.java, Ejercicio1.java (Examen30Abril)

1.8 ALGORITMOS IMPORTANTES

Ordenación burbuja sobre ArrayList

```
for (int i = 0; i < lista.size()-1; i++) {
    for (int j = 0; j < lista.size()-1-i; j++) {
        if (lista.get(j).getValor() > lista.get(j+1).getValor()) {
            MiClase temp = lista.get(j);
            lista.set(j, lista.get(j+1));
            lista.set(j+1, temp);
        }
    }
}
```

→ Ejemplo: Tarea.java (ListaTareas) — ordenar por prioridad

Factorial

```
long fact = 1;
for (int i = 1; i <= n; i++) { fact *= i; }
// Recursivo:
long factorial(int n) { return (n <= 1) ? 1 : n * factorial(n-1); }
```

Número aleatorio

```
(int)(Math.random() * n) // entero entre 0 y n-1
(int)(Math.random() * 6) + 1 // dado: entero entre 1 y 6
// Con clase Random:
Random rnd = new Random();
int x = rnd.nextInt(n); // entre 0 y n-1
```

→ Ejemplos: Buscaminas.java, Juego.java (Examen26Feb), Ejercicio_1.java (Examen27Nov)

1.9 VALIDACIÓN DE ENTRADA

```
Scanner sc = new Scanner(System.in);

// Validar número entero
int num = -1;
boolean valido = false;
while (!valido) {
    try {
        System.out.print('Introduce un número: ');
        num = Integer.parseInt(sc.nextLine().trim());
        valido = true;
    } catch (NumberFormatException e) {
        System.out.println('Error: introduce un número válido.');
```

```
}

// Validar rango
while (num < 1 || num > 100) {
System.out.print('Fuera de rango (1-100): ');
num = Integer.parseInt(sc.nextLine().trim());
}

// Validar email básico
boolean emailValido = email.contains('@') && email.contains('.');

// Validar contraseña (match)
String p1, p2;
do {
p1 = sc.nextLine(); p2 = sc.nextLine();
} while (!p1.equals(p2));
```

→ Ejemplos: Boletin_3 (contraseña), Buscaminas.java (try-catch), Validar_Correo.java

SECCIÓN 2 — ÍNDICE DE PROYECTOS

Proyecto	Clases principales	Conceptos clave
BADMINTON	Persona(abs), Arbitros, Jugadores, Entrenadores, Clientes, Tienda	Herencia, TreeSet, ArrayList, HashSet, LocalDate, categoría por edad
BANCO	Banco, Sucursal, Clientes, Cuentas	Composición, ArrayList, constructor overloading, IBAN
CONCESIONARIO	Vehiculo(abs), Coche, Moto, Conductor	Herencia, abstract class, composición
TRANSPORTES	Persona(abs), Conductor, Cliente, Vehiculo, Envio, Ruta, Empresa	Herencia, ArrayList, toString() override
FÚTBOL	Persona(abs), Jugador, Equipo, Entrenador, Árbitro, Partido	Herencia, HashSet, estadísticas, getters/setters
MÉDICOS	Persona(abs), Médico, Paciente, Cita, Especialidad	Herencia, ArrayList, LocalDate, DateTimeFormatter
PROTECTORA	Animales(abs), Perros, Gatos, Tortugas, Clientes, Protectora	Herencia, constructor overloading, abstract class
GESTIÓN PERSONA 3ªEV	Persona implements Baja/Jubilacion/Accidente	Interfaces múltiples, LocalDate, Period, constantes
LAMBDA	Operacion(@FunctionalInterface), Main, Producto	@FunctionalInterface, expresiones lambda
LISTA TAREAS	Tarea, main	ArrayList estático, ficheros, burbuja, static members
MANEJO FICHEROS	ficheros, escrituraFicheros, ficherosBinarios, Tarea	BufferedReader, Files API, PrintWriter, Serialización
BASES DE DATOS	BasesDeDatos, EjercicioLogin, BBDD4	JDBC, PreparedStatement, ResultSet, SQLException
FESTIVAL (EX26)	Artista(abs), Cantante, GrupoMusica, Caseta	Abstract, calcularSueldo(), static counter, ART-001
AEROPUERTO (EX26)	Personal, Piloto, AuxiliarVuelo, Vuelo	ArrayList + .contains() duplicados, búsqueda por código
CARNAVAL (EX26)	Agrupacion, Chirigota, Cupletista, Sesion	Herencia, jerarquía de clases
JJOJ	Jugador, Deporte(abs), DeporteIndiv., DeporteEquipo	Herencia, abstract class, composición
OOP FINAL (Pokémon)	Pokemon, Combate, orientadaObjetos	OOP completa, sistema de combate
EX. 27 NOV (Datos)	Ejercicio_1, _2, _3	Arrays, Math.random(), estadísticas, validación
EX. 26 FEB (Juego)	Juego, Jugadores, Pruebas_juego	ArrayList, .remove(index), Math.random()
EX. 27 ENERO (Empl.)	Empleados(abs), Programadores, Jefes_proyecto, Proyecto	Abstract, static counter, String.format() para códigos
EX. 30 ABRIL (BBDD)	Ejercicio1, Ejercicio4, Rectangulo	JDBC, PreparedStatement, ResultSet con parámetros
EX. AUTOESCUELA	Preguntas, Respuestas, Examen	OOP, sistema de test/quiz

SECCIÓN 3 — BOLETINES DE EJERCICIOS

Boletín	Ejercicios	Tema principal	Conceptos
Boletín 1	22	Bucles básicos	for, while, do-while, System.out.println(), aritmética
Boletín 2	1+	Strings	compareTo(), if-else, Scanner, orden alfabético
Boletín 3	1+	Validación entrada	while, equals(), control de flujo, input validation
Boletín 4	1+	Matemáticas	Factorial, for loop, tipo long, operaciones
Boletín 13	varios	Clases y herencia	Persona, Alumno, Profesor, Modulo — herencia, override
Boletín 23	11	Ficheros (I/O)	BufferedReader, FileReader, try-with-resources, split()
Boletín 24	9	Avanzado	Temas avanzados varios

SECCIÓN 4 — EJERCICIOS PARA EXAMEN

Ejercicio	Técnicas utilizadas
Contador de vocales	Arrays + bucles anidados + charAt() + toLowerCase()
Buscaminas	Array 2D + Math.random() + try-catch para validación de entrada

Validar correo	String.contains('@') + String.contains('.') + lógica condicional
Validar contraseña	Reglas de longitud, charAt(), Character.isDigit/isLetter, bucle while
Palíndromos	StringBuilder.reverse() o bucle inverso + .equals()
Composición cadenas	StringBuilder.append() / concatenación con +
Mezclar caracteres	charAt(), substring(), String.valueOf(), manipulación de posiciones
Generador matrículas	String.format(), Math.random(), (char)('A'+rnd)
Adivinar número	Math.random(), bucles, Integer.parseInt(), try-catch, contador
Array con validación	Arrays, comprobaciones de rango, conteo de elementos
Análisis votos	Arrays o HashMap, toLowerCase(), trim(), isEmpty(), null-check
Cajero automático	OOP, ArrayList, Scanner, validación de operaciones

SECCIÓN 5 — PLANTILLAS RÁPIDAS DE CÓDIGO

Abstract class + Herencia completa

```
public abstract class Persona {
    private String nombre;
    public Persona(String nombre) { this.nombre = nombre; }
    public String getNombre() { return nombre; }
    public abstract String getDatos();
}

public class Alumno extends Persona {
    private String ciclo;
    public Alumno(String nombre, String ciclo) { super(nombre); this.ciclo = ciclo; }
    @Override
    public String getDatos() { return getNombre() + ' - ' + ciclo; }
}
```

Interface con constante + implementación

```
public interface Jubilacion {
    int EDAD_JUBILACION = 65;
    void consultaJubilacion();
}

public class Persona implements Jubilacion {
    @Override
    public void consultaJubilacion() {
        int edad = Period.between(fechaNac, LocalDate.now()).getYears();
        System.out.println(edad >= EDAD_JUBILACION ? 'Jubilado' : 'No jubilado');
    }
}
```

ArrayList con control de duplicados (como en Aeropuerto)

```
ArrayList<AuxiliarVuelo> auxiliares = new ArrayList<>();

public void asignarAuxiliar(AuxiliarVuelo aux) {
    if (!auxiliares.contains(aux)) {
        auxiliares.add(aux);
    } else {
        System.out.println('Ya asignado');
    }
}

public AuxiliarVuelo buscarAuxiliar(String codigo) {
    for (AuxiliarVuelo a : auxiliares) {
        if (a.getCodigo().equals(codigo)) return a;
    }
    return null;
}
```

Leer objetos desde fichero de texto

```
// Suponiendo formato CSV: 'nombre,edad,ciudad'
try (BufferedReader br = new BufferedReader(new FileReader('datos.txt'))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        String[] partes = linea.split(',');
        String nombre = partes[0];
        int edad = Integer.parseInt(partes[1]);
        String ciudad = partes[2];
        lista.add(new Persona(nombre, edad, ciudad));
    }
} catch (IOException e) { e.printStackTrace(); }
```

Lambda con @FunctionalInterface

```
@FunctionalInterface
public interface Operacion { int ejecutar(int a, int b); }

Operacion suma = (a, b) -> a + b;
Operacion resta = (a, b) -> a - b;
Operacion multi = (a, b) -> a * b;
System.out.println(suma.ejecutar(5, 3)); // 8
System.out.println(multi.ejecutar(4, 2)); // 8
```

JDBC con PreparedStatement — plantilla completa

```
public static void consultarPorCiudad(String ciudad) {
    String url = 'jdbc:mysql://localhost/empresa';
    String user = 'root';
    String pass = '1234';
    try (Connection con = DriverManager.getConnection(url, user, pass);
        PreparedStatement ps = con.prepareStatement(
            'SELECT nombre, salario FROM empleados WHERE ciudad = ?')) {
        ps.setString(1, ciudad);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            System.out.println(rs.getString('nombre') +
                ' - ' + rs.getDouble('salario'));
        }
    } catch (SQLException e) { e.printStackTrace(); }
}
```

Burbuja sobre ArrayList de objetos

```
// Ordenar lista de Tarea por prioridad (menor primero)
for (int i = 0; i < tareas.size()-1; i++) {
    for (int j = 0; j < tareas.size()-1-i; j++) {
        if (tareas.get(j).getPrioridad() > tareas.get(j+1).getPrioridad()) {
            Tarea temp = tareas.get(j);
            tareas.set(j, tareas.get(j+1));
            tareas.set(j+1, temp);
        }
    }
}
```

HashMap para contar votos

```
HashMap<String, Integer> votos = new HashMap<>();
String[] candidatos = {'Ana', 'Luis', 'María'};
for (String c : candidatos) votos.put(c, 0);

String voto = sc.nextLine().trim().toLowerCase();
if (!voto.isEmpty() && votos.containsKey(voto)) {
    votos.put(voto, votos.get(voto) + 1);
}
// Mostrar resultados:
for (String k : votos.keySet()) {
    System.out.println(k + ': ' + votos.get(k));
}
```